

Bootstrap 5 Grid System & Utilities

A Complete, Beginner-Friendly & Professional Class

1 INTRODUCTION

◆ What is Bootstrap?

Bootstrap is a **CSS framework** that helps us build **responsive, mobile-first websites faster**. It provides:

- Prebuilt layout system (Grid)
- Ready utility classes
- Consistent design patterns

Think of Bootstrap as **LEGO blocks for websites** ☐.

◆ Why the Grid System Exists

Before Bootstrap:

- Layouts were messy
- Different screens broke designs
- Too much custom CSS

The Grid System solves:

- ✓ Alignment
 - ✓ Responsiveness
 - ✓ Consistency
-

◆ Why Utilities Are Important

Utilities are **small helper classes** that:

- Add spacing
- Control display
- Align content
- Change colors & text quickly

Instead of writing custom CSS, you **reuse utilities**.

◆ Real-World Analogy 🏠

Imagine building an apartment:

```
Building (container)
├── Floor (row)
│   └── Rooms (columns)
```

Bootstrap Grid works the **same way**.

2 BOOTSTRAP GRID SYSTEM (DEEP EXPLANATION)

◆ container & container-fluid

🧠 Explanation

- `.container` → fixed width (centered)
- `.container-fluid` → full screen width

📊 Diagram

```
.container
|         content         |

.container-fluid
|-----|
```

🏠 Analogy

- Container = Apartment
- Container-fluid = Open field

✓ Example 1

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<div class="container bg-light p-3">
  Fixed container
</div>
```

✓ Example 2

```
<div class="container-fluid bg-warning p-3">
  Full width container
</div>
```

📱📱📱 Responsive behavior: both adapt automatically.

◆ row

🔗 Explanation

Rows **group columns** and remove side padding.

📊 Diagram

```
| row |  
| col | col | col |
```

🏠 Analogy

Row = Floor of a building

✓ Example

```
<div class="container">  
  <div class="row">  
    <div class="col bg-primary text-white">Column 1</div>  
    <div class="col bg-secondary text-white">Column 2</div>  
  </div>  
</div>
```

◆ Column System (12 Columns)

🔗 Explanation

Bootstrap divides each row into **12 equal parts**.

📊 Diagram

```
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
```

🏠 Analogy

Pizza 🍕 cut into 12 slices.

✓ Example 1

```
<div class="row">  
  <div class="col-6 bg-success">50%</div>  
  <div class="col-6 bg-info">50%</div>  
</div>
```

✓ Example 2

```
<div class="row">  
  <div class="col-4 bg-danger">33%</div>  
  <div class="col-8 bg-dark text-white">66%</div>  
</div>
```

◆ Responsive Columns (col-sm, col-md, etc.)

🔗 Explanation

Change layout based on screen size.

📊 Breakpoints

```
xs <576px
sm ≥576px
md ≥768px
lg ≥992px
xl ≥1200px
xxl ≥1400px
```

✓ Example 1

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-4 bg-primary">Box</div>
  <div class="col-12 col-md-6 col-lg-8 bg-secondary">Box</div>
</div>
```

📱 Mobile: stacked

💻 Desktop: side by side

◆ Auto Layout Columns

🔗 Explanation

Bootstrap auto divides space equally.

✓ Example

```
<div class="row">
  <div class="col bg-info">Auto</div>
  <div class="col bg-warning">Auto</div>
  <div class="col bg-danger">Auto</div>
</div>
```

◆ Nested Grid

🔗 Explanation

You can place rows inside columns.

📊 Diagram

```
Row
├── Col
│   ├── Row
│   │   └── Col
```

✓ Example

```
<div class="row">
  <div class="col-6 bg-light">
    Parent
    <div class="row">
      <div class="col-6 bg-info">Child</div>
      <div class="col-6 bg-warning">Child</div>
    </div>
  </div>
</div>
```

◆ Alignment

🔗 Explanation

Use flex utilities inside rows.

✓ Example

```
<div class="row align-items-center" style="height:150px">
  <div class="col bg-success">Centered vertically</div>
</div>
```

◆ Ordering Columns

🔗 Explanation

Change order without changing HTML.

✓ Example

```
<div class="row">
  <div class="col order-2 bg-danger">Second</div>
  <div class="col order-1 bg-success">First</div>
</div>
```

◆ Offsets

🔗 Explanation

Leave empty space before columns.

✓ Example

```
<div class="row">
  <div class="col-4 offset-4 bg-primary">Centered</div>
</div>
```

3 BOOTSTRAP UTILITIES (FULL COVERAGE)

◆ Spacing Utilities (m & p)

🔧 Problem Solved

Adds margin & padding quickly.

□ Syntax

```
m-3 → margin  
pt-4 → padding-top  
mx-auto → horizontal center
```

✗ Bad

```
<div style="margin:20px;"></div>
```

✓ Good

```
<div class="m-3 p-4 bg-light"></div>
```

◆ Display Utilities

```
<div class="d-none d-md-block">Visible on md+</div>
```

◆ Flex Utilities

```
<div class="d-flex justify-content-between">  
  <div>Left</div>  
  <div>Right</div>  
</div>
```

◆ Text Utilities

```
<p class="text-center text-uppercase fw-bold">Hello</p>
```

◆ Color & Background

```
<div class="bg-dark text-white p-3">Dark Box</div>
```

◆ Width & Height

```
<div class="w-50 h-100 bg-info">50% width</div>
```

◆ Position Utilities

```
<div class="position-relative">
  <span class="position-absolute top-0 end-0">X</span>
</div>
```

◆ Visibility

```
<div class="invisible">Hidden but space remains</div>
```

4 CODE WEAK POINTS ✕

✕ Missing row

```
<div class="container">
  <div class="col-6"></div>
</div>
```

✓ Fix

```
<div class="container">
  <div class="row">
    <div class="col-6"></div>
  </div>
</div>
```

5 BEST PRACTICES ✓

- Use **Grid for layout**, Flex for alignment
 - Do NOT overuse utilities
 - Use utilities for **small adjustments**
 - Keep HTML readable
 - Test on mobile first 📱
-

6 BEFORE vs AFTER

✕ Bad

```
<div style="float:left;width:50%"></div>
```

✓ Better

```
<div class="col-6"></div>
```

```
<div class="col-12 col-md-6"></div>
```

7 MINI PRACTICE TASKS ☐

- 1 Create a 3-column responsive layout
- 2 Center a card vertically & horizontally
- 3 Build a mobile-first navbar grid
- 4 Create nested grid gallery
- 5 Hide content on mobile only

💡 Hint: Think **container** → **row** → **col**

8 QUICK CHEAT SHEET

◆ Breakpoints

Class	Screen
col	all
col-sm	≥576px
col-md	≥768px
col-lg	≥992px

◆ Common Utilities

- m-3, p-4
- d-flex
- text-center
- bg-primary

9 FINAL SUMMARY

Key Takeaways

- Grid = structure
- Utilities = quick control
- Mobile first always
- Clean HTML = professional code

☐ Mental Model

```
Container
├── Row
│   └── Column
```

If you master this, **Bootstrap becomes easy forever** 